

离散数学

刘晓芳

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

第9章 树



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

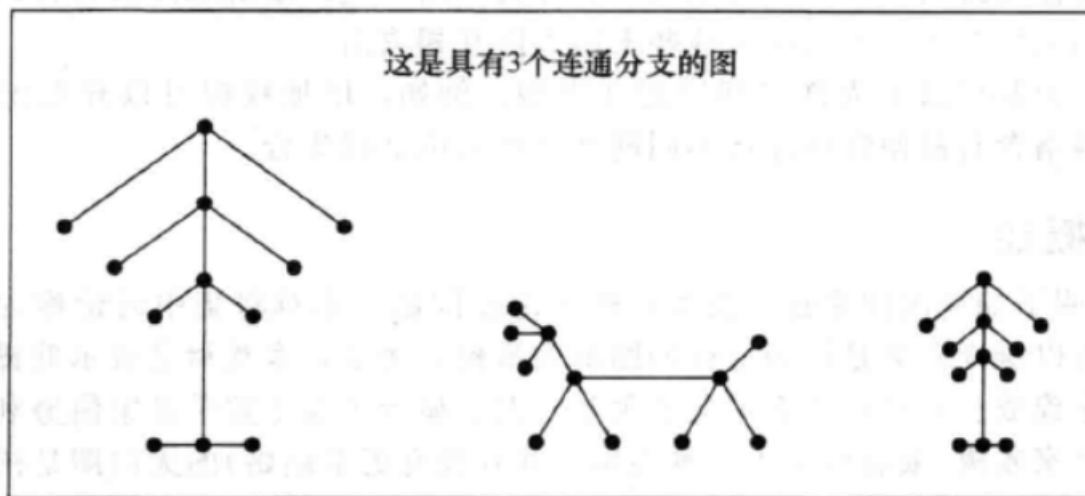
主要内容

➤ 第9章 树

- 9.1 定义
- 9.2 最小生成树
- 9.3 根树
- 9.4 前缀码
- 9.5 遍历算法
- 9.6 前缀记法和后缀记法

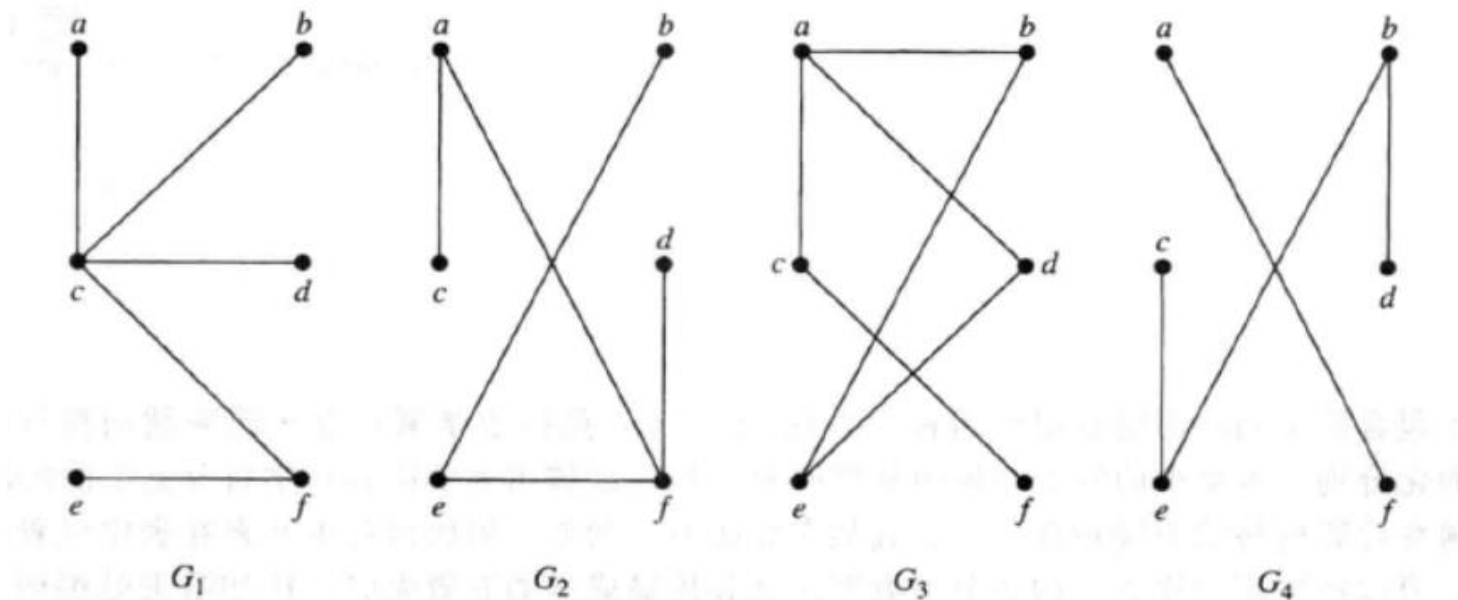
9.1 定义

- 定义1：不含回路的连通无向图称为无向树，简称树。每个连通分支均是树的非连通无向图称为森林。平凡图称为平凡树。树中度为1的顶点称为树叶，度数大于等于2的顶点称为分支点。
- *因为树中无圈，因此，树中没有环和多重边。



9.1 定义

➤ 例1：图2中哪个图是树？



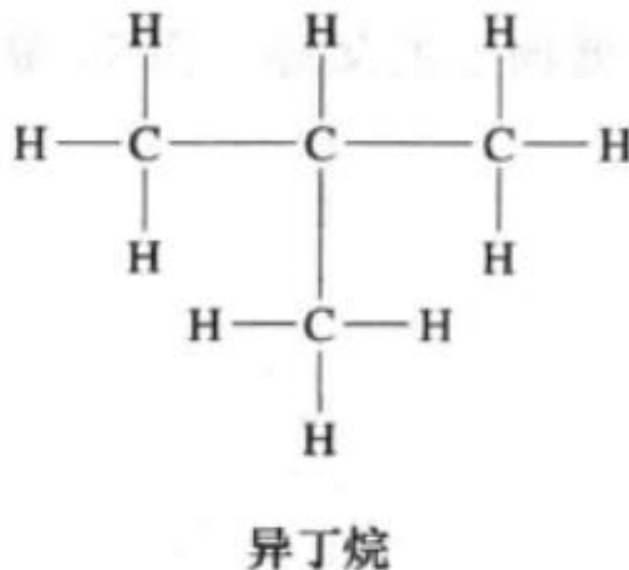
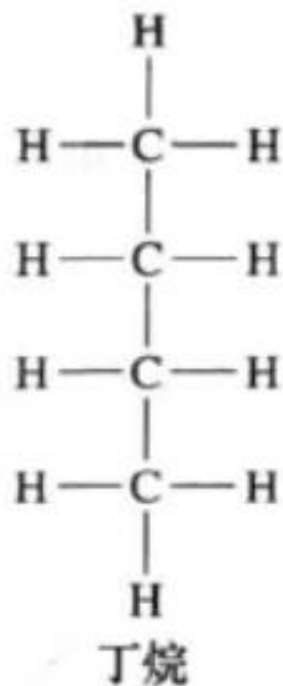
➤ 解： G_1 和 G_2 是树，因为它们都是连通无圈的。 G_3 不是树，因为 e, b, a, d, e 是一个圈。最后， G_4 不是一棵树，因为它不连通。

9.1 定义

- 例5：饱和碳氢化合物和树：树是1857年英国数学家Cayley研究饱和碳氢化合物 C_nH_{2n+2} 的同分异构体时发现的。
- 在 C_nH_{2n+2} 的分子结构中，碳原子被表示成一个度为4的顶点，而氢原子被表示成一个度为1的顶点。该图有 $3n + 2$ 个顶点，由握手定理，该图有 $\frac{4n+2n+2}{2} = 3n + 1$ 条边。由于该图是连通图，并且有 $3n+1$ 条边， $3n+2$ 个顶点(由后面介绍树的性质知)，该图是一棵树。具有以上性质不同构的树是 C_nH_{2n+2} 的同分异构体。例如： C_4H_{10} 有两种同分异构体，见图。

9.1 定义

- C_4H_{10} 有两种同分异构体



9.1 定义

- 例6：表示一个组织结构：一个电脑公司的组织结构见图10。每个顶点的组织的负责人，为它的父结点的负责人工作。例如：硬件开发的负责人，为研发部门的副总经理工作。



9.1 定义

- 定理1：设 $G=\langle V, E \rangle$ ， $|V|=n$ ， $|E|=m$ ，则下面个命题是等价的：
 - (1) G 连通且不含回路
 - (2) G 的每对顶点之间有唯一的一条路径
 - (3) G 是连通的且 $m=n-1$
 - (4) G 中无回路且 $m=n-1$
 - (5) G 中无回路，但在任意两个不相邻的顶点之间增加一条边，就形成唯一的一条初级回路
 - (6) G 是连通的且每条边都是桥

9.1 定义

- 定理2: n 阶非平凡的树中至少有两片树叶
- 证明: 非平凡树的每个顶点的度数都大于等于1, 设有 k 片树叶, 则有 $n-k$ 个顶点度数大于等于2。由握手定理,

$$2m \geq k + 2(n-k)$$

- 有定理1, $m=n-1$,代入上式得

$$2(n-1) \geq k + 2(n-k)$$

- 整理得

$$k \geq 2$$

9.1 定义

- 定义2：设 $G=\langle V, E \rangle$ 是无向连通图， T 是 G 的生成子图并且是树，则称 T 是 G 的生成树。 G 在 T 中的边称为 T 的树枝，不在 T 的边称为 T 的弦。 T 的所有弦的集合的导出子图称为 T 的余树。

9.1 定义

- 定理3：任何无向连通图都有生成树
- 证明：设 G 是一个无向连通图，如果 G 中无回路，则 G 本身是树，当然也是 G 的生成树。如果 G 中存在回路 C ，删除 C 上任意一条边，这不影响图的连通性。如果还有回路，就再删除回路上的一条边，直至没有回路为止，最后得到一个连通无回路的生成子图，它是 G 的生成树。

9.1 定义

- 推论：设 n 阶无向连通图 G 有 m 条边，则 $m \geq n-1$
- 证明：根据定理3， G 有生成树，生成树中有 $n-1$ 条边，从而 $m \geq n-1$

9.1 定义

- 定义3: 设 T 是连通图 $G=\langle V, E \rangle$ 的一颗生成树, 对每一条弦 e , 存在唯一的由弦 e 和 T 的树枝构成的初级回路 C_e , 称 C_e 为对应于弦 e 的基本回路, 称所有基本回路的集合为对应生成树 T 的基本回路系统。
- 基本回路的个数等于 $m-n+1$, 其中 n 是连通图的顶点数, m 是边数。

9.1 定义

- **定义4：** 设 T 是连通图 $G=\langle V, E \rangle$ 的一棵生成树，对 T 的每一条树枝 a ，存在唯一的由树枝 a ，其余的边都是弦的割集 S_a ，称 S_a 为对应树枝 a 的基本割集，称所有割集的集合为对应生成树 T 的基本割集系统。

9.1 定义

- 定义5：设 C_1 和 C_2 是两条回路，如果 C_1 和 C_2 有一条或几条共同的边，那么删去共同的边，每一条回路变成若干条通路，这些通路可以连接成一条或几条回路，称作合作 C_1 和 C_2 。合并的结果记为 $C_1 \oplus C_2$ 。
- 定义6：连通图中任一条回路都可以表示成基本回路的合并。任一条回路必有弦，它等于对应所含弦的基本回路的合并。
- 2个广义割集的合并等于它们的对称差。

9.2 最小生成树

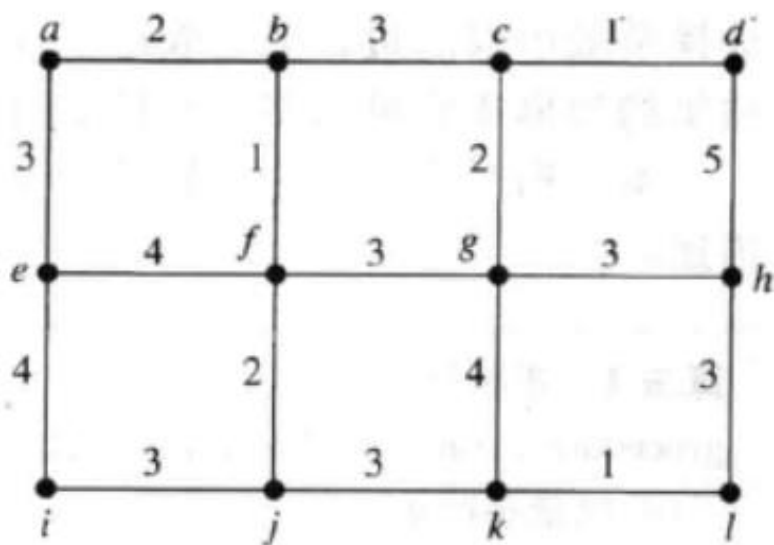
- 定义7: 设 G 是一个简单图。 G 的一棵生成树是 G 的一个子图, 它是一棵树, 并且包含 G 的所有顶点。
- 最小生成树: 设 T 是无向连通带权图 $G=\langle V, E, W \rangle$ 的生成树, T 中所有边的权之和称为 T 的权, 记作 $W(T)$ 。 G 中所有生成树中权最小的生成树称为最小生成树。
- 最小生成树问题: 求任给的无向连通带权图的最小生成树。
- *注意: 一个图的生成树可能不是唯一的(举例说明)

9.2 最小生成树

- 求最小生成树的算法：每次取权尽可能小的边，只要它不与已取的边不构成回路，直到取得一棵生成树为止。这就是避圈法（Kruskal算法）。
- 输入： n 阶无向连通带权图 $G=\langle V, E, W \rangle$ ，其中 $E=\{e_1, \dots, e_m\}$
- 输出： G 的最小生成树 T
- (1)按权大小对边排序
- (2)令 $T \leftarrow \emptyset$ ， $i \leftarrow 1$ ， $k \leftarrow 0$
- (3)若 e_i 与 T 中的边不构成回路，则令 $T \leftarrow T \cup \{e_i\}$ ， $k \leftarrow k+1$ 。
- (4)若 $k < n-1$ ，则令 $i \leftarrow i+1$ ，转(3)

9.2 最小生成树

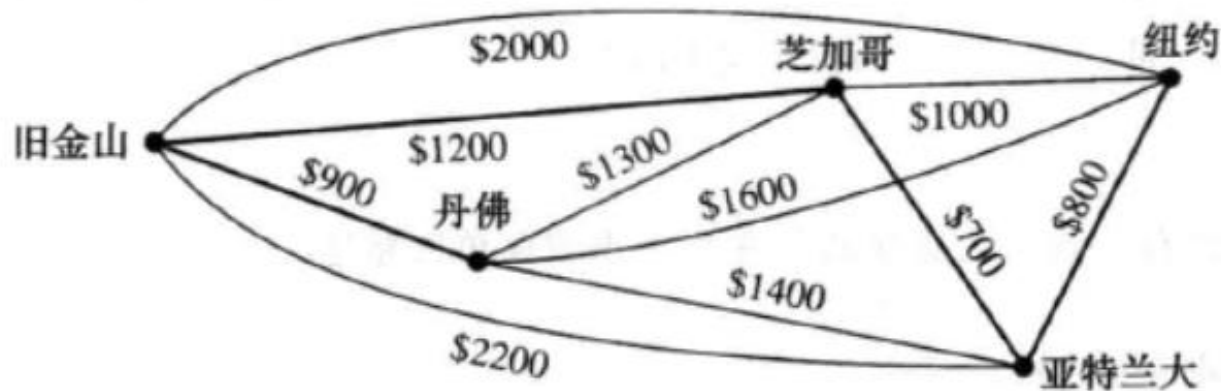
➤ 例子：用Kruskal算法求最小生成树



选择	边	权
1	{c, d}	1
2	{k, l}	1
3	{b, f}	1
4	{c, g}	2
5	{a, b}	2
6	{f, j}	2
7	{b, c}	3
8	{j, k}	3
9	{g, h}	3
10	{i, j}	3
11	{a, e}	3
总计:		24

9.2 最小生成树

- *一个计算机公司要建立一个通讯网络连接各计算中心，任意一对中心之间至少有一条电话线路相连，并且使得该网络的总代价最小。这个问题可以通过求赋权图的最小生成树来解决。



9.3 根树

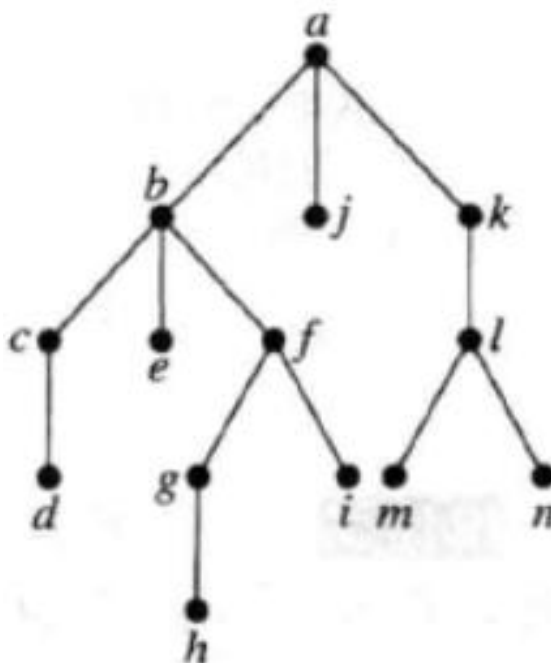
- 定义2: 一个有向图 D , 如果略去有向边的方向所得无向图为一棵无向树, 则称 D 为有向树。
- 根树(rooted tree)是一棵非平凡有向树, 如果一个顶点入度为0, 其余顶点入度均为1。其中入度为0的顶点称为树根, 入度为1、出度为0的顶点称为树叶, 入度为1、出度大于0的顶点称为内点, 内点和树根统称为分支点。
- *同一棵树, 选不同的顶点作为树根, 该有根树就是不同的。

9.3 根树

- 根树 T 中，从树根到顶点 v 的通路长度称为 v 的层数，记为 $l(v)$ ，称层数相同的顶点在同一层。顶点的最大层数称为 T 的树高，记为 $h(T)$ 。

9.3 根树

- 例10：给出图中所有顶点的层数和树高。
- 解：树根a层数为0；顶点b,j,k层数为1；顶点c,e,f,l层数为2；顶点d,g,i,m和n层数为3；最后，顶点h层数为4。因为层数最大的顶点的层数为4，故树高为4。

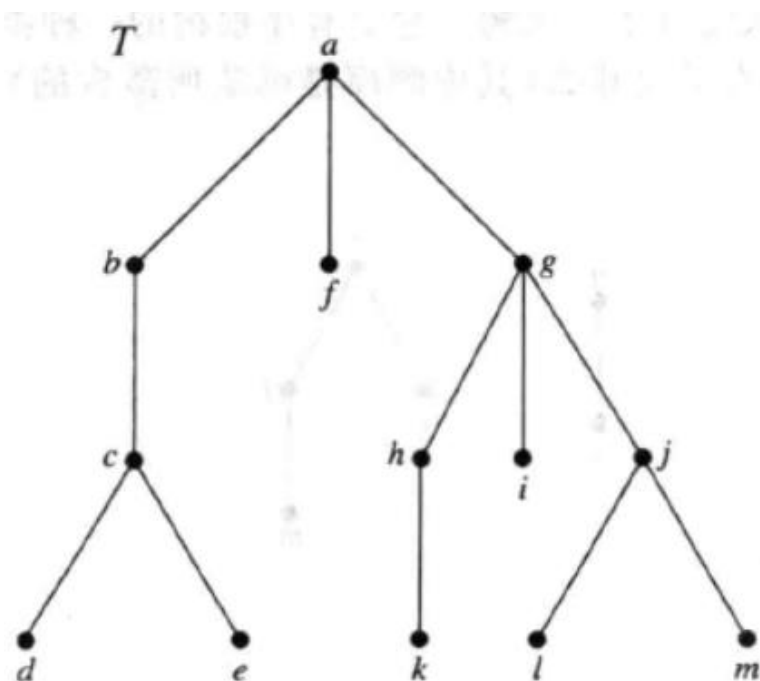


9.3 根树

- 一棵根树可以看成一棵家族树：
- *父亲(parent):
- *孩子(child):
- *兄弟(sibling):
- *祖先(ancestor):
- *后代(descendant):
- *树叶(leaf):
- *内部顶点(internal vertex):
- *子树(subtree):

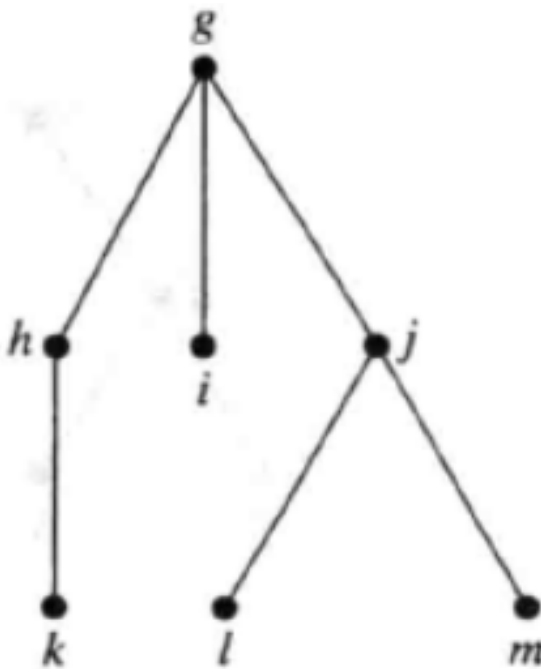
9.3 根树

- 例2：在图5中，给定有根树 T (树根为 a)，求 c 的父亲， g 的孩子， h 的兄弟， e 的所有祖先， b 的所有后代，所有内部顶点，和所有树叶。g为根的子树是什么？



9.3 根树

- 解：c的父亲是b；g的孩子是h,i和j；h的兄弟是i和j；e的祖先是c,b和a；b的后代是c,d和e；该树的所有内部顶点是：a,b,c,g,h和j；该树的所有树叶是：d,e,f,i,k,l和m；以g为根的子树如图。



9.3 根树

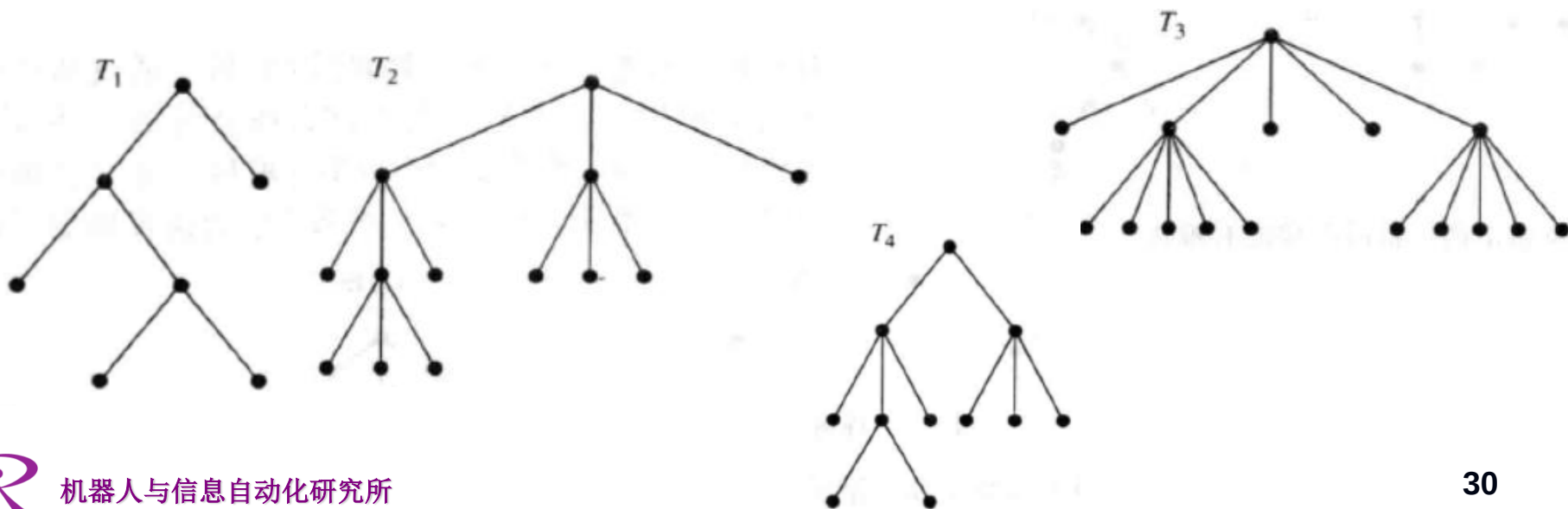
- 定义6：设 T 为一棵根树， a 为 T 中一个顶点且不是根树，称 a 及其后代导出的子图 T' 为 T 的以 a 为根的子树，简称根子树。
- 定义7：如果对根树每一层上的顶点规定次序，称这样的根树为有序树。
- 次序可标在顶点处，或者边上（从左到右，从上到下）

9.3 根树

- 定义8：设 T 为一棵根树，
- (1)若 T 的每个分支点至多有 r 个儿子，则称 T 为 r 叉树
- (2)若 T 的每个分支点恰好有 r 个儿子，则称 T 为 r 叉正则树
- (3)若 T 是 r 叉正则树，且所有树叶的层数都等于树高，则称 T 为 r 叉完全正则树
- (4)若 r 叉树是有序的，则称 T 为 r 叉有序树
- (5)若 r 叉正则树是有序的，则称 T 为 r 叉正则有序树
- (6)若 r 叉完全正则树 T 是有序的，则称 T 是 r 叉完全正则有序树。

9.3 根树

- 例3：图7中的树是否 m 叉正则树(m 为某个正整数)？
- 解： T_1 是二叉正则树， T_2 是3叉正则树， T_3 是5叉正则树， T_4 不是 m 叉正则树，因为某些内部顶点有2个孩子，而某些内部顶点有3个孩子。



9.3 根树

- 定义9: 设2叉树 T 有 t 片树叶 $v_1, v_2, \dots, v_t$, 权分别为 $w_1, w_2, \dots, w_t$, 称 $W(T) = \sum_{i=1}^t w_i l(v_i)$ 为 T 的权, 其中 $l(v_i)$ 是树叶 v_i 的层数。在所有 t 片树叶, 其权为 $w_1, w_2, \dots, w_t$ 的2叉树中, 权最小的2叉树称作权为 $w_1, w_2, \dots, w_t$ 的最优2叉树。

9.4 前缀码

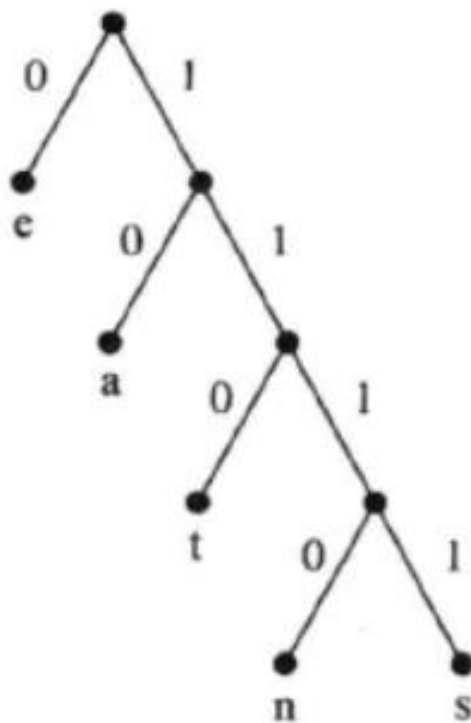
- 考虑这样的问题：我们要给英语字母表中的每一个字母用0,1串编码(假设不区别大小写)，那么我们可以用长度为5的0,1串给每个字母编码(因为长度为5的0,1串有32个不同的串)。我们的问题是能否用更少的0,1字符表示一个字母串？这样我们可以节省存储和通讯中的传输时间。
- 如果我们知道英语字母中，每个字母出现的频率是不同的，那么，我们可以用较短的0,1串表示出现频率高的字母，用较长的0,1串表示出现频率低的字母，使得整个英语字母串的0,1串的长度尽可能小。

9.4 前缀码

- 例如：我们可以用0表示e，用1表示a，用01表示t，但这样，位串0101就可能表示eat, tea, eaea, 或tt等不同的串，因而出现混淆。
- 为此，我们引入前缀码(prefix codes)来表示英语字母。
- 前缀：设 $\beta = a_1a_2 \cdots a_n$ 是长度为n的字符串，称其子串 $a_1a_2 \cdots a_j$ ($1 \leq j \leq n$) 是 β 的前缀。
- *前缀码：每一个字母的位串不是另一个字母的位串的前缀。

9.4 前缀码

- 例如：在图5中，我们用一个二叉树来定义字母 e, a, t, n, s 的前缀码。任一字符串 11111011100 被唯一地确定为字母串 “sane”。



9.4 前缀码

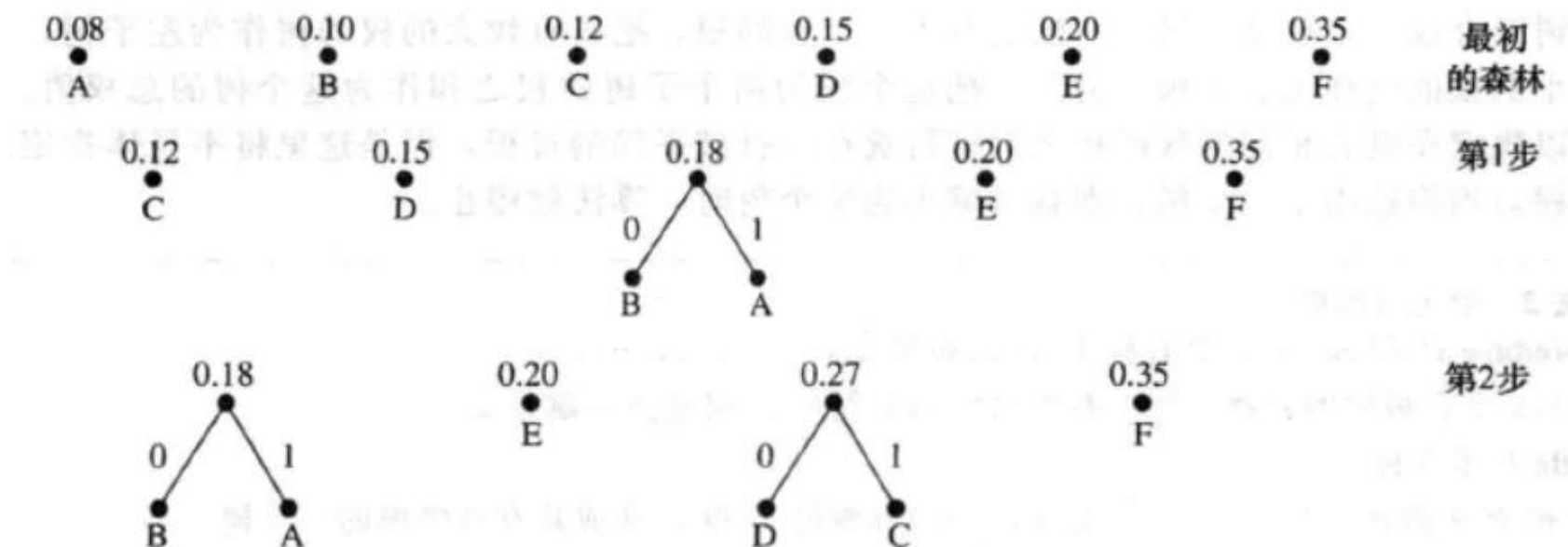
- *Huffman编码：如果我们知道字母表中每个字母出现的频率，那么我们可以构造一个最优树。从该树得到每一个字母的前缀码，使得在以上所说的出现频率下，整个串的二进制位数最少。

9.4 前缀码

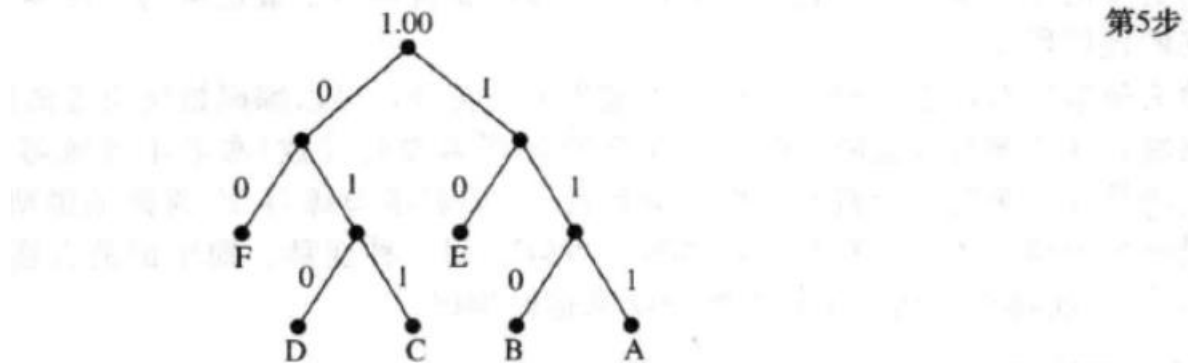
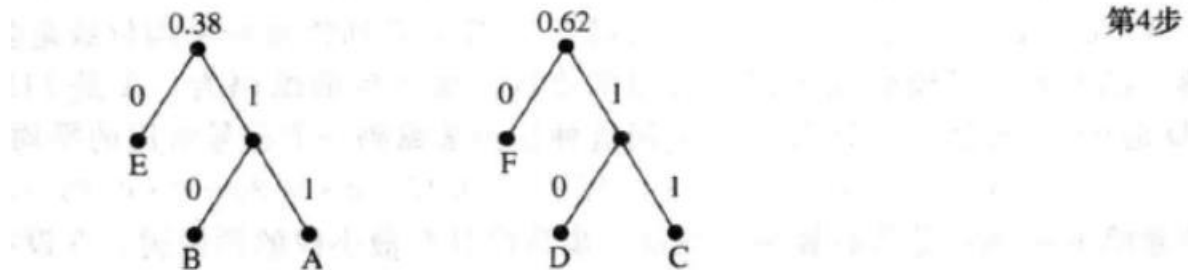
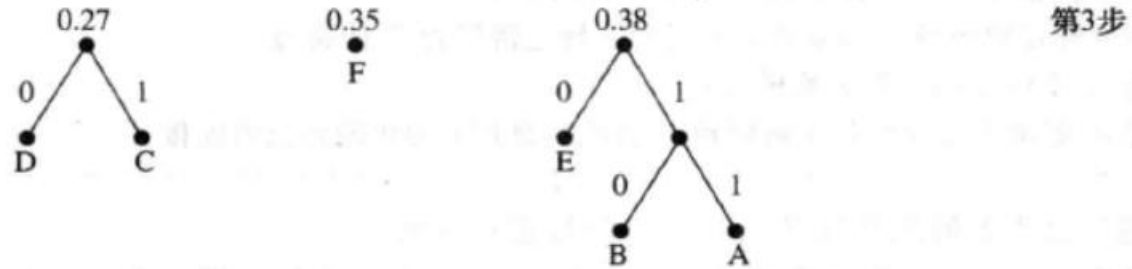
- 算法2: Huffman编码
- PROCEDURE Huffman (C: 一个表, 里面每一个字母 a_i 的出现频率为 w_i , $i = 1, 2, \dots, n$);
- $F := n$ 个有根树的森林, 每个有根树只含一个顶点 a_i , 它的权是 w_i ;
- WHILE F 不是一棵树 DO
- BEGIN
- 在 F 中选两个权最小的有根树 T 和 T' 满足 $w(T) \geq w(T')$, 建立新的根 r , r 的左子树是 T , r 的右子树是 T' , r 与 T 相连的边标记为0, r 与 T' 相连的边标记为1;
- 将 $w(T) + w(T')$ 作为新的树的权, 用新的树取代 F 中的 T 和 T' ;
- END;
- {每个符号 a_i 的Huffman编码是从最后 F 的树的根结点到标记为 a_i 的叶子的唯一路径上各边的标记构成的串}

9.4 前缀码

- 例5：使用Huffman编码给以下字母编码，其中，每个字母出现的频率是：
A: 0.08; B: 0.10; C: 0.12; D: 0.15; E: 0.20; F: 0.35;。
- 解：构造Huffman树的过程见图。



9.4 前缀码



9.4 前缀码

➤ 各字母的编码:

A: 111; B: 110; C: 011; D: 010; E: 10; F: 00。

➤ 使用这种编码来编码一个符号所用的平均位数是

$$3 \times 0.08 + 3 \times 0.10 + 3 \times 0.12 + 3 \times 0.15 + 2 \times 0.20 + 2 \times 0.35 = 2.45$$

➤ *如果传输 10^n ($n \geq 2$)个按上述频率出现的字母, 需要多少位二进制数字? 若用等长的(长度为3的)码子传输需要多少个二进制位?

➤ 传输100个字母需要: $3 \times 8 + 3 \times 10 + 3 \times 12 + 3 \times 15 + 2 \times 20 + 2 \times 35 = 245$ 位二进制位。

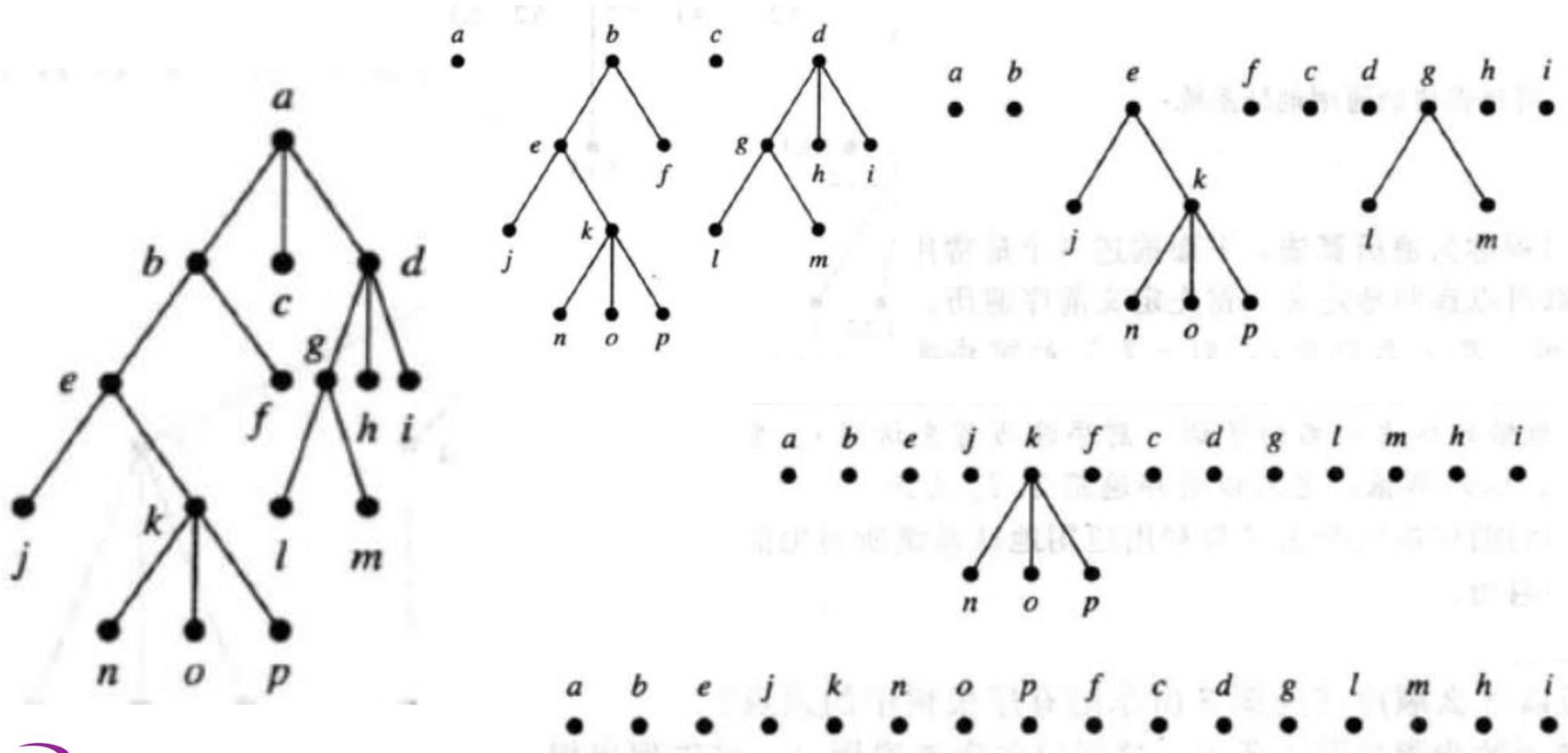
➤ 传输 10^n 个字母需要: $10^{n-2} \times 245 = 2.45 \times 10^n$ 个二进制位。而用等长编码需要 3×10^n 个二进制位。

9.5 遍历算法

- *我们有3种遍历算法，即前序遍历(preorder traversal)，中序遍历(inorder traversal)和后序遍历(postorder traversal)。
- 定义1：设 T 是一个有序根树，树根为 r 。如果 T 只含顶点 r ，则 r 是前序遍历的顺序。否则，假设 $T_1, T_2, \dots, T_n$ 是 r 的子树按从左到右的顺序列出，前序遍历 T ，首先访问 r ，然后前序遍历 T_1 ，再前序遍历 $T_2, \dots$ ，最后，前序遍历 T_n 。

9.5 遍历算法

➤ 例2：图4中树的前序遍历顶点顺序是什么？

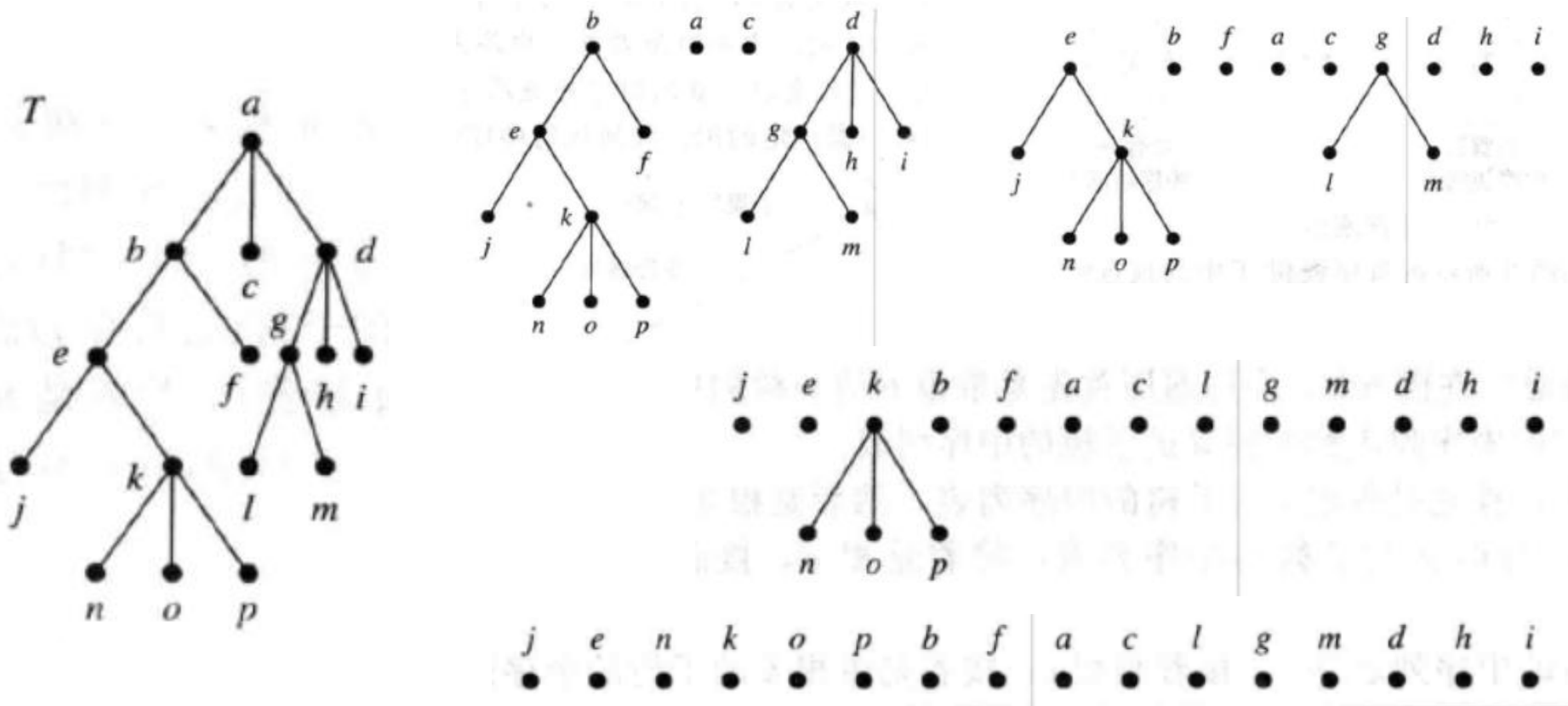


9.5 遍历算法

- 定义2：设 T 是一个有序有根树且树根为 r 。如果 T 只含有 r ，那么 r 是 T 的中序遍历的顺序。否则，假设 r 的子树从左到右列出为 $T_1, T_2, \dots, T_n$ ，那么 T 的中序遍历首先中序遍历 T_1 ，然后访问 r ，再中序遍历 T_2 ，中序遍历 T_3 ， $\dots$ ，最后中序遍历 T_n 。

9.5 遍历算法

➤ 例3：图6中的树的中序遍历的顺序是什么？

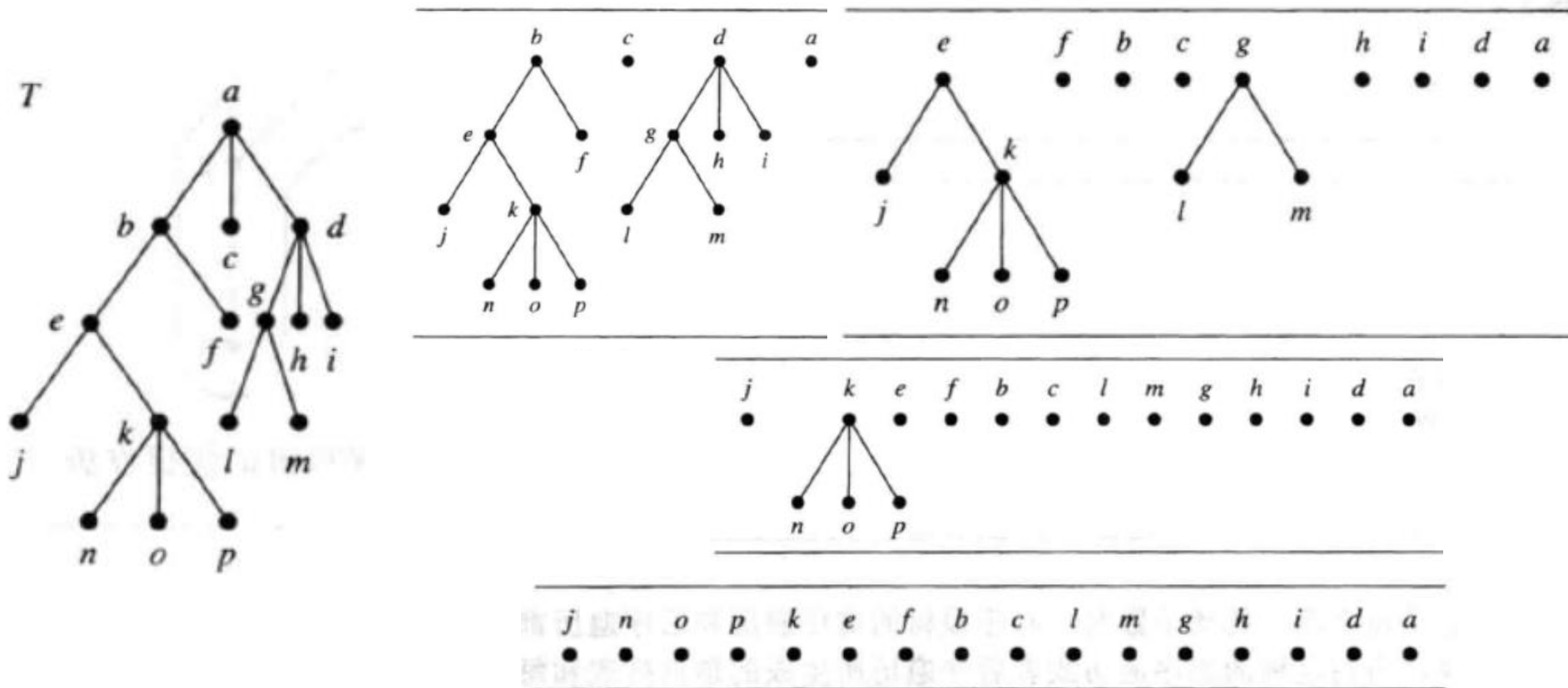


9.5 遍历算法

- 定义3：设 T 是有序根树且树根为 r 。如果 T 只包含 r ，那么 r 是 T 的后序遍历的顺序。否则，假设 r 的子树从左到右列出顺序为 $T_1, T_2, \dots, T_n$ 。那么 T 的后序遍历顺序为，先后序遍历 T_1 ，再后序遍历 $T_2, \dots$ ，再后序遍历 T_n ，然后访问 r 。

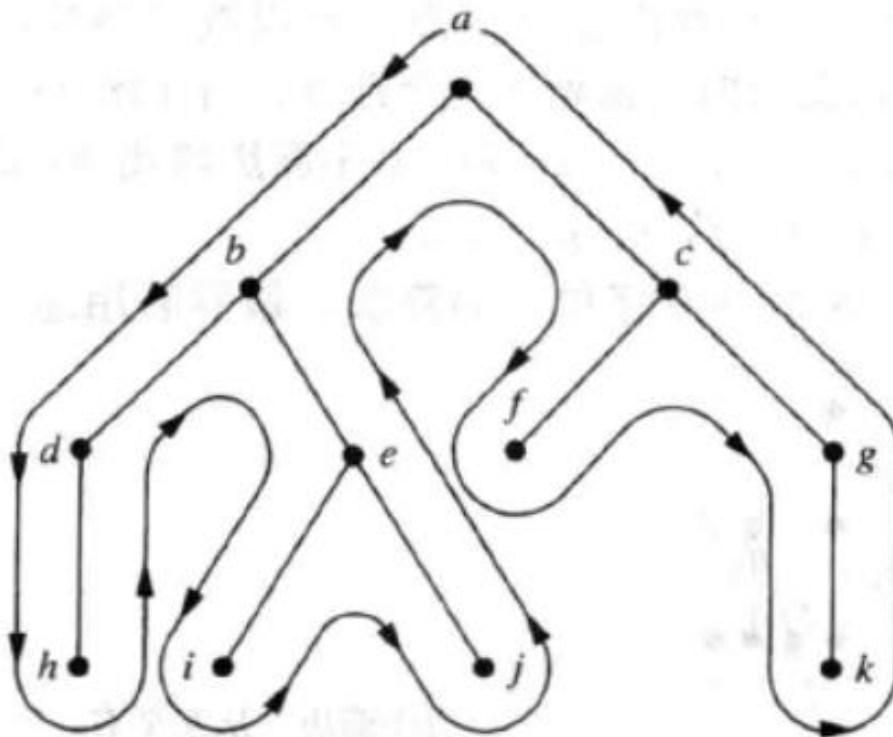
9.5 遍历算法

➤ 例子：图8中的树后序遍历的顺序是什么？



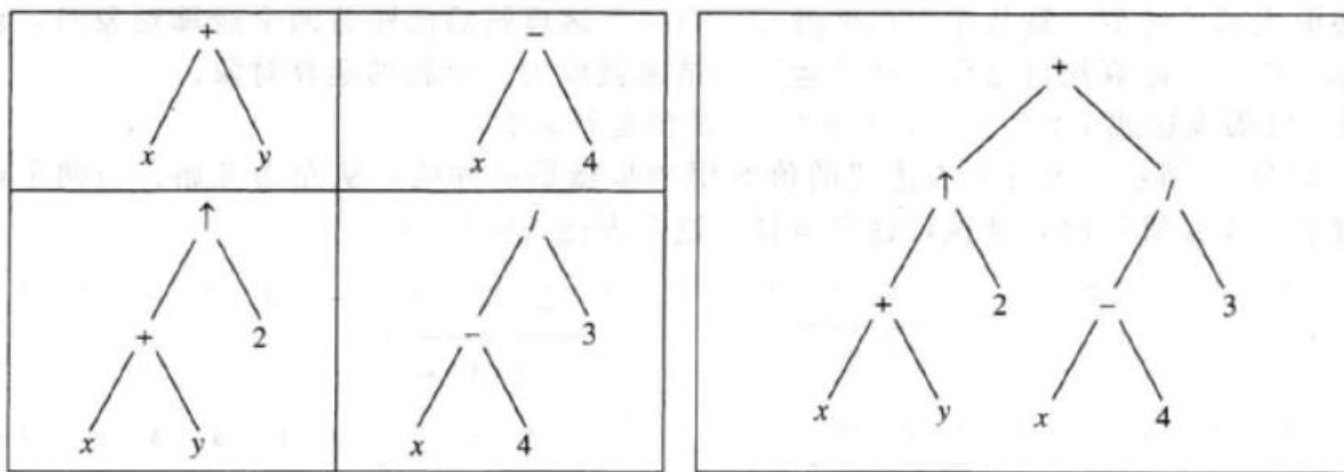
9.5 遍历算法

- *前序遍历、中序遍历和后序遍历的顶点的顺序，可以按一定的规律直接列出。



9.6 前缀和后缀记法

- 有序树表示复杂的表达式：分支点放运算符。
- 被减数和被除数放在左子树，对一元运算符，分支点只有一个儿子，运算对象是以儿子为根的根子树表示的子表达式。数字和变量放在树叶上。
- 例子: $((x+y) \uparrow 2) + ((x-4)/3)$



9.6 前缀和后缀记法

- **前缀形式**: $+ \uparrow +xy2 / -x \ 4 \ 3$, 写成前缀形式的表达式称为**前缀符号法或波兰符号法**
- 从右向左步骤求表达式的值, 并用右边的运算对象来执行运算。
- **后缀形式**: $xy+2 \ \uparrow x4-3 / +$, 写成后缀形式的表达式称为**后缀符号法或逆波兰记法** (无二义性的, 不需要括号)
- 从左向右进行, 当一个运算符后面跟着两个运算对象时, 执行运算。在一个运算执行后, 运算的结果就称为一个新的运算对象。

9.6 前缀和后缀记法

- 例7. 前缀表达式 $+ - * 2 3 5 / \uparrow 2 3 4$ 的值是什么?

$$\begin{array}{ccccccccccc}
 + & - & * & 2 & 3 & 5 & / & \uparrow & 2 & 3 & 4 \\
 & & & & & & & \underbrace{} & & & \\
 & & & & & & & 2 \uparrow 3 = 8 & & & \\
 + & - & * & 2 & 3 & 5 & / & 8 & 4 & & \\
 & & & & & & & \underbrace{} & & & \\
 & & & & & & & 8 / 4 = 2 & & & \\
 + & - & * & 2 & 3 & 5 & 2 & & & & \\
 & & & \underbrace{} & & & & & & & \\
 & & & 2 * 3 = 6 & & & & & & & \\
 + & - & 6 & 5 & 2 & & & & & & \\
 & & \underbrace{} & & & & & & & & \\
 & & 6 - 5 = 1 & & & & & & & & \\
 + & 1 & 2 & & & & & & & & \\
 & \underbrace{} & & & & & & & & & \\
 & 1 + 2 = 3 & & & & & & & & &
 \end{array}$$

表达式的值: 3

9.6 前缀和后缀记法

➤ 例8. 后缀表达式 $7\ 2\ 3\ *\ -\ 4\ \uparrow\ 9\ 3\ /\ +$ 的值是多少?

$$7\ 2\ 3\ *\ -\ 4\ \uparrow\ 9\ 3\ /\ +$$

$$2 * 3 = 6$$

$$7\ 6\ -\ 4\ \uparrow\ 9\ 3\ /\ +$$

$$7 - 6 = 1$$

$$1\ 4\ \uparrow\ 9\ 3\ /\ +$$

$$1^4 = 1$$

$$1\ 9\ 3\ /\ +$$

$$9 / 3 = 3$$

$$1\ 3\ +$$

$$1 + 3 = 4$$

表达式的值: 4

作业29

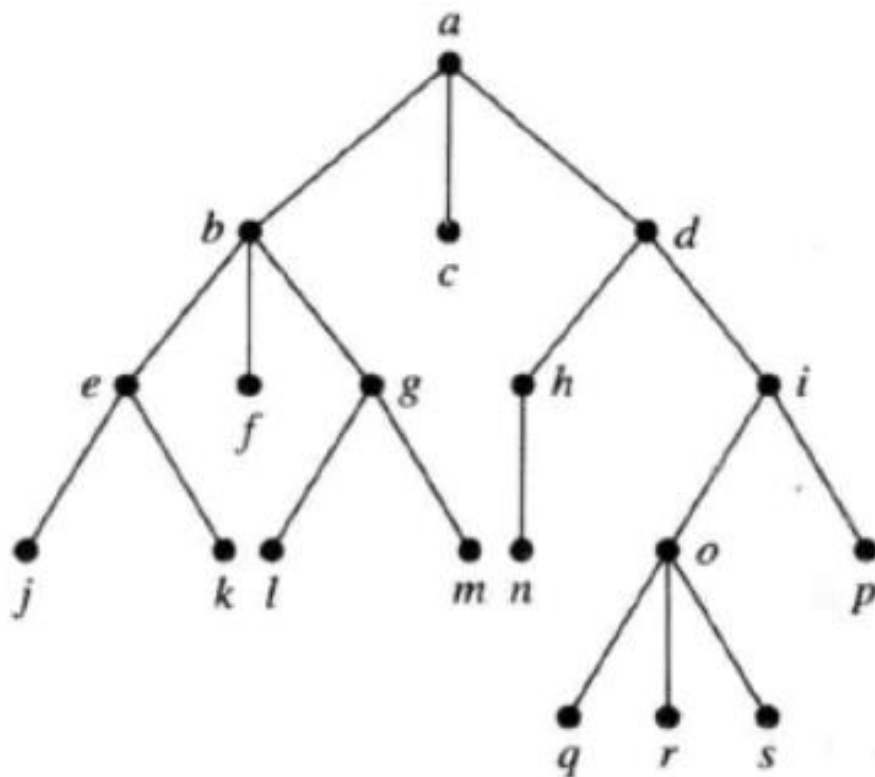


- 1. 一棵无向树 T 有5片树叶，3个2度顶点，其余顶点都是3度顶点，问 T 有几个顶点？
- 2. 证明：一个简单图是树当且仅当它是连通的，并且删除其中任意一条边所得到的图是不连通的。
- 3. 假设1000个人参加象棋比赛，每比一场一定有一人赢，一人输，输的人被淘汰，用一个有根树来做模型，问要决定冠军，需要多少场比赛？

作业29



- 4. 给出树的前序遍历、中序遍历和后序遍历的顶点序列。



作业29



- 5. 设7个字母在通讯中的出现的频率如下：
a: 35% , b: 20% , c: 15% , d: 10% , e: 10% ,
f: 5% , g: 5%
- 用Huffman算法求传输它们的前缀码。要求画出最优树，指出每个字符的编码，并指出传输 10^n 个按上述频率出现的字母，需要多少个2进制数字。
- 6. 一个算式用逆波兰符号法表示为：
 $ab+cd+e**fgh*--+$ ，试画出该算式的二叉有序树表示，并写出该算式的波兰法符号表示。
- 7. 无向图 $G=<V,E>$ 。证明G是树的充要条件是G的每对顶点之间有唯一路径。

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University